System Programming Project 3

담당 교수 : 김영재 교수님

이름 : 김민우

학번 : 20212019

1. 개발 목표

이번 프로젝트의 주요 목표는 Concurrent Stock Server를 설계하고 구현하는 것이다. 이는 여러 클라이언트가 동시에 접속하여 주식 정보를 조회, 구매, 판매하는 작업을 수행할 수 있게 한다. 구체적인 사항들은 다음과 같다.

- 주식 서버 기능 구현 : 주식 정보를 저장하고 여러 클라이언트와 통시하여 주식 정보 조회, 구매, 판매 기능을 제공한다. 이때 서버는 주식 목록을 관리하고 클라이언트의 요청에 따라 주식 정보를 실시간으로 업데이트하고, 남은 수량을 관리한다.
- 클라이언트 기능 구현 : 각 클라이언트는 서버에 접속해 주식을 사거나 팔고, 가격과 재고를 조회하는 등의 요청을 할 수 있어야한다. 또한 서버와의 통신을 통해 요청한 정보를 실시간으로 받아볼 수 있어야 한다.
- 동시성 처리 : 'select()' 시스템 호출을 사용하여 여러 클라이언트의 요청을 비동기적으로 처리하는 Event기반 접근과 'pthread'라이브러리를 사용하여 여러 클라이언트 요청을 병렬로 처리하는 Thread기반 접근을 사용하여 각각 서버를 구현한다.
- 성능 평가 및 분석 : Event기반 접근과 Thread기반 접근 방식의 성능을 평가하고 분석한다. 예를들어, 모든 클라이언트가 buy또는 sell요청을 하는 경우와 show요청을 섞어서 하는 경우 등을 비교한다.

개발 범위 및 내용

A. 개발 범위

- 아래 항목을 구현했을 때의 결과를 간략히 서술

1. Task 1: Event-driven Approach

Task 1은 stockserver를 특정 포트 번호로 실행하고, stockclient 또는 multiclient를 사용하여 명령어 (show, buy, sell, exit)를 처리한다. 서버는 클라이언트의 요청에 따라 결과를 클라이언트에 반환하며, 받은 명령어의 바이트 수와 연결된 connfd 번호를 기록한다. 이 과정에서는 select() 시스템 호출을 활용하여 여러 클라이언트 연결을 비동기적으로 관리하게 된다. 이를 통해 서버는 여러 클라이언트의 요청을 동시에 효율적으로 처리할 수 있다.

2. **Task 2: Thread-based Approach**

Task 2는 stockserver를 특정 포트 번호로 실행하고, stockclient 또는 multiclient를 사용하여 명령어 (show, buy, sell, exit)를 처리한다. 서버는 클라이언트의 요청에 따라 결과를 클라이언트에 반환하며, 받은 명령어의 바이트 수와 연결된 connfd 번호를 기록한다. 이 접근 방식에서는 각 클라이언트의 요청을 개별 쓰레드에서 병렬로 처리하게 된다. pthread 라이브러리를 사용하여 쓰레드를 생성하고, 동기화 문제를 해결하기 위해 P 및 V 함수를 사용하여 데이터를 보호한다.

3. **Task 3: Performance Evaluation**

Event 접근 방식과 Thread 접근 방식으로 구현된 서버를 성능 측면에서 비교 평가한다. 이를 위해 multiclient 프로그램을 수정하여 다양한 클라이언트 수와 명령어 유형에 따른 성능을 측정한다. 시간 측정 함수를 사용하여 각 접근 방식의 동시 처리율과 응답 시간을 평가하고, 두 접근 방식의 장단점을 비교한다.

B. 개발 내용

- **아래 항목의 내용만 서술**
- **(기타 내용은 서술하지 않아도 됨. 코드 복사 붙여 넣기 금지)**
- **Task1 (Event-driven Approach with select())**

✓ Multi-client 요청에 따른 I/O Multiplexing 설명

connfd 관리를 위해 pool을 초기화한다. 이 초기화 단계에서는 pool 구조체의 변수들을 설정하고, 클라이언트 파일 디스크립터 배열을 초기화한다. 반복문을 통해 서버의 listenfd를 확인하고, 이를 pool의 ready_set에 추가하여 클라이언트 연결 요청을 대기한다. 매 반복마다 Select 함수를 호출하여 준비된 클라이언트 요청 수를 파악하고, 활성화된 클라이언트를 식별한다. 이를 통해 활성화된 클라이언트의 파일 디스크립터를 찾아내어 해당 클라이언트의 요청을 처리하는 함수를 실행게 된다. 클라이언트로부터 받은 요청은 show, buy, sell, exit 등의 명령어로 구분되며, 각 명령어에 따라 적절한 처리를 수행한다. 예를 들어, show 요청은 현재 주식 목록을 반환하고, buy와 sell 요청은 주식 데이터를 수정한다. 처리된

결과는 클라이언트에게 다시 전송되고 모든 요청 처리가 완료되면, 주식 데이터의 변화를 파일에 저장하여 지속적으로 데이터의 최신 상태를 유지한다. 이러한 과정은 서버가 여러 클라이언트의 요청을 효율적으로 처리하고, 일관된 주식 정보를 관리할 수 있도록 한다.

✓ epoll과의 차이점 서술

Select는 선형 검색 알고리즘을 사용하기 때문에 많은 수의 소켓을 처리할 때 성능이 저하된다. 이는 모든 소켓을 순차적으로 검사하기 때문이다. 반면, epoll은 커널 이벤트 기반 메커니즘을 사용하여 이벤트가 발생한 소켓만 처리하기 때문에 더 효율적이다. epoll은 수천 개 이상의 파일 디스크립터를 효율적으로 관리할 수 있으며, CPU 자원을 적게 소모한다. 또한, epoll은 파일 디스크립터의 상태를 커널에 저장하고 필요 시 가져와 처리하여 자원 낭비를 줄인다. 그러나 epoll은 리눅스에서만 지원되며, select는 대부분의 운영 체제에서 사용 가능하다. 결과적으로, epoll은 대규모 네트워크 프로그램에서 더 나은 성능을 제공하지만, 리눅스에서만 사용할 수 있다는 한계가 있다.

- **Task2 (Thread-based Approach with pthread)**

✓ Master Thread의 Connection 관리

Master Thread는 Accept 함수를 호출하여 클라이언트 연결을 받아들인다. 클라이언트 수는 전역 변수로 관리되며, 이를 위해 Semaphore를 사용하여 동기화 작업을 수행한다. 클라이언트가 연결되면 Pthread_create 함수를 이용해 새로운 스레드를 생성하여 서버와 클라이언트 간의 통신을 처리한다. 이 과정에서 Semaphore를 사용하여 스레드 간의 동기화 문제를 해결하고, 자원 접근을 조율한다.

✓ Worker Thread Pool 관리하는 부분에 대해 서술

각 스레드는 반복적으로 클라이언트 요청을 모니터링하고 처리한다. 요청을 처리하는 동안, Semaphore를 사용하여 동기화를 유지하고, 읽기 및 쓰기 작업의 충돌을 방지한다. 스레드가 작업을 수행할 때마다 Semaphore는 자원 접근을 제어하여 데이터의 일관성을 보장한다. 이 방식은 여러 스레드가 동시에 동일한 자원에 접근하지 않도록 하여 시스템의 안정성을 높인다. 따라서 서버는 클라이언트 요청을 효율적으로 처리하고, 성능을 극대화할 수 있다.

- **Task3 (Performance Evaluation)**

✓ 얻고자 하는 metric 정의, 그렇게 정한 이유, 측정 방법 서술

측정하고자 하는 주요 지표는 클라이언트 수가 증가함에 따라 Task1(Event-driven Approach)과 Task2(Thread-based Approach)의 동시 처리율 변화를 분석하는 것이다. 특히, buy, sell, show 명령어를 처리할 때 동시 처리율이 클라이언트 수에 따라 어떻게 달라지는지 확인하고자 한다. 이를 통해 Task1과 Task2의 성능을 클라이언트 수와 명령어 유형에 따라 비교하며, 각 작업이 얼마나 많은 시간을 소요하는지 파악할 수 있다. 예를 들어, 10, 50, 100, 500명의 클라이언트가 동시에 요청을 보낼 때 각 접근 방식의 처리 성능을 평가한다. 또한, 각 명령어(buy, sell, show)가 클라이언트 수에 따라 처리 시간이 어떻게 변하는지도 비교한다. 이러한 분석을 통해 가장 시간이 많이 소요되는 작업과 그 이유를 확인할 수 있다.

클라이언트 요청의 처리 시간을 측정하기 위해 sys/time.h의 gettimeofday 함수를 사용한다. 프로그램 시작 시 gettimeofday로 시작 시간을 기록하고, 여러 클라이언트 프로세스를 생성하여 서버에 요청을 보낸 후 응답을 받는다. 각 클라이언트는 랜덤하게 show, buy, sell 명령어를 생성하여 서버에 전송한다. 모든 클라이언트 요청이 완료되면, 다시 gettimeofday로 종료 시간을 기록한다. 시작 시간과 종료 시간의 차이를 계산하여 전체 처리 시간을 마이크로초 단위로 출력한다. 이를 통해 클라이언트 수와 명령어 유형에 따른 Task1과 Task2의 성능을 비교할 수 있다.

✓ Configuration 변화에 따른 예상 결과 서술

Task1에서는 클라이언트 수가 증가해도 서버의 기본 설정이 크게 변하지 않기 때문에, 일정한 성능을 유지할 가능성이 높다. 이 접근 방식은 비동기 I/O 처리를 통해 클라이언트 요청을 효과적으로 관리한다. 예를 들어, show, buy, sell 명령어를 처리할 때 높은 처리율을 보일 수 있다. 그러나 최대 연결 가능한 클라이언트 수에는 한계가 있어, 클라이언트 수가 매우 많아지면 성능이 떨어질 수 있다. 궁극적으로, 클라이언트 수가 많을 경우 Task2보다 성능이 더 우수할 수 있다.

반면 Task2는 클라이언트 수가 늘어남에 따라 생성되는 스레드 수도 증가한다. 클라이언트 수가 많아질수록 스레드 관리가 복잡해지고, 컨텍스트 스위칭 오버헤드가 발생할 수 있습니다. 처음에는 처리율이 증가하지만, 일정 수준을 넘어서면 성능 저하가 발생할 가능성이 크다. 병렬 처리 덕분에 클라이언트 수가 적을 때는 show, buy, sell 명령어의 처리 속도가 빨라진다. 하지만 스레드 수가 너무 많

아지면 오버헤드로 인해 성능 저하가 발생할 수 있다.

C. 개발 방법

1. Task1

- **클라이언트 관리를 위한 Pool 구조체 정의 및 초기화**

먼저, 다수의 클라이언트 연결을 관리하기 위해 pool 구조체를 정의한다. 이 구조체는 여러 클라이언트의 파일 디스크립터와 각 클라이언트의 읽기 상태를 관리하기 위해 사용된다. pool 구조체는 최대 파일 디스크립터 번호, 읽기 파일 디스크립터 집합, 준비된 파일 디스크립터 집합, 준비된 파일 디스크립터 수, 클라이언트 파일 디스크립터 배열, 클라이언트의 읽기 상태를 저장하는 rio_t 구조체 배열로 구성된다. init_pool 함수는 pool 구조체를 초기화한다. 초기화 과정에서는 pool 구조체의 변수들을 설정하고, 클라이언트 파일 디스크립터 배열을 초기화한다. 서버의 listenfd를 확인하고 이를 pool의 read_set에 추가하여 클라이언트 연결 요청을 대기하도록 설정한다.

- **클라이언트 추가 및 제거 함수**

클라이언트가 연결될 때 이를 pool에 추가하는 add_client 함수를 구현했다. 이 함수는 새로운 클라이언트의 파일 디스크립터를 pool의 clientfd 배열에 저장하고, 읽기 파일 디스크립터 집합에 추가한다. 추가된 클라이언트의 최대 파일 디스크립터 번호와 배열 인덱스를 업데이트한다. 반대로, 클라이언트 연결이 종료될 때 이를 pool에서 제거하는 remove_client 함수도 구현했다. 이 함수는 해당 클라이언트의 파일 디스크립터를 닫고, 읽기 파일 디스크립터 집합에서 제거하여 더 이상 해당 클라이언트가 관리되지 않도록 한다.

- **클라이언트 요청 처리 함수**

클라이언트로부터 들어오는 요청을 처리하는 handle_clients 함수를 구현했다. 이 함수는 select 시스템 호출을 사용하여 준비된 클라이언트 요청을 식별하고 처리합니다. select는 여러 파일 디스크립터를 동시에 모니터링하여 하나 이상의 파일 디스크립터가 읽기 가능 상태가 될 때 반환한다. 반환된 후에는 준비된 클라이언트의 요청을 처리하고, 각 클라이언트가 보낸 명령어를 분석하여 적절한 작업을 수행한다. 명령어는 show, buy, sell, exit 등으로 구분되며, 각각의 명령어에 따라 주식 목록을 반환하거나 주식 데이터를 수정한다. 요청이 처리되면 클라이언트에게 결과를 전송하고, 모든 요청 처리가 완료되면 주식 데이터의 변화를 파일에

저장하여 최신 상태를 유지한다.

- **메인 함수 수정**

메인 함수에서는 클라이언트 연결 요청을 받아들이고, 이를 pool에 추가하여 관리한다. 먼저 서버 소켓을 초기화하고, pool을 초기화한다. 이후 무한 루프를 통해 select 시스템 호출을 사용하여 클라이언트 요청을 처리한다. 새로운 클라이언트 연결 요청이 들어오면 이를 pool에 추가하고, 클라이언트로부터 받은 요청을 처리하는 함수를 호출하여 클라이언트 명령을 처리한다. 모든 클라이언트 요청이 처리된 후에는 주식 데이터를 파일에 저장한다.

2. Task2

- **Worker Thread Pool 관리**

Master Thread는 클라이언트의 연결 요청을 받아들이고 새로운 Worker Thread를 생성한다. 이를 위해 Accept 함수를 호출하여 클라이언트 연결을 받아들인 후, 전역 변수 active_clients로 클라이언트 수를 관리한다. active_clients의 변경은 세마포어인 state_mutex를 사용하여 동기화한다. 클라이언트가 연결되면 Pthread_create 함수를 이용해 새로운 Worker Thread를 생성하여 클라이언트의 요청을 처리한다. 이 과정에서 세마포어를 사용하여 스레드 간의 동기화 문제를 해결하고 자원 접근을 조율한다. 예를 들어, 클라이언트 수를 증가시키거나 감소시킬 때 state_mutex 세마포어를 사용하여 동기화한다.

- **Worker Thread의 요청 처리**

각 Worker Thread는 반복적으로 클라이언트 요청을 모니터링하고 처리한다. 클라이언트의 요청을 처리하는 동안, 세마포어를 사용하여 동기화를 유지하고, 읽기 및 쓰기 작업의 충돌을 방지한다. Worker Thread는 Rio_readlineb 함수를 사용하여 클라이언트로부터 요청을 읽고, handle_cmd 함수를 호출하여 요청을 처리한다. 요청이 show, buy, sell, exit 등의 명령어로 구분되며, 각각의 명령어에 따라 적절한 처리를 수행한다. show 요청은 현재 주식 목록을 반환하고, buy와 sell 요청은 주식 데이터를 수정한다. 이 과정에서 각 주식 항목의 mutex 세마포어를 사용하여 동기화하고, 주식 데이터의 일관성을 유지한다. 클라이언트와의 통신이 종료되면 state_mutex 세마포어를 통해 active_clients를 안전하게 감소시킨다.

- **주식 데이터의 동기화 및 저장**

주식 데이터를 수정할 때는 각 주식 항목에 대해 세마포어를 사용하여 동기화한다. 예를 들어, process_stock 함수에서 stock->data.mutex 세마포어를 사용하여 주식 데이터를 수정하는 작업을 동기화한다. 이를 통해 여러 스레드가 동시에 주식 데이터를 수정할 때 발생할 수 있는 충돌을 방지한다. 모든 요청 처리가 완료되면 save_stock 함수를 호출하여 주식 데이터의 변경 사항을 파일에 저장한다. 이를 통해 데이터의 일관성을 유지하고, 시스템의 안정성을 높일 수 있다.

2. 구현 결과

- **2번의 구현 결과를 간략하게 작성**

전체적으로 Task1은 주식 재고를 관리하는 서버를 구현하며, 클라이언트로부터 명령을 받아 주식을 조회, 구매, 판매하는 기능을 수행한다. 주식 데이터는 이진 탐색 트리(BST)에 저장되고, 서버가 시작될 때 파일에서 로드된다. 각 주식 항목은 ID, 재고 수량(left_stock), 가격(price)으로 구성되며, 이러한 항목들은 노드(node) 구조체로 관리된다. 서버는 클라이언트의 연결을 효율적으로 관리하기 위해 클라이언트 풀(pool)을 사용하며, select() 시스템 호출을 통해 여러 클라이언트로부터 들어오는 데이터를 비동기적으로 처리한다. 클라이언트는 show, buy <ID> <수량>, sell <ID> <수량> 명령을 보낼 수 있다. show 명령은 모든 주식 항목을 조회하여 클라이언트에게 보내고, buy <ID> <수량> 명령은 특정 주식 항목을 구매하며, 재고 수량이 충분하지 않으면 에러 메시지를 반환한다. sell <ID> <수량> 명령은 특정 주식 항목을 판매합니다.

서버는 클라이언트의 요청에 따라 주식 데이터를 실시간으로 수정하며, 모든 클라이언트가 연결을 종료하면 주식 데이터를 파일에 저장한다. 새로운 클라이언트가 접속하면 클라이언트 풀에 추가되고, 클라이언트가 명령을 보내지 않고 연결을 종료하면 클라이언트 풀에서 제거된다. 서버는 안정적으로 주식 데이터를 관리하고 여러 클라이언트와의 동시 접속을 처리하며, 모든 명령에 대한 응답을 클라이언트에게 제공하여 실시간으로 주식의 조회, 구매, 판매를 수행할 수 있다.

Task2는 멀티스레드 환경에서 주식 재고를 관리하는 서버를 구현하며, 클라이언트로부터 명령을 받아 주식을 조회, 구매, 판매하는 기능을 수행한다. 주식 데이터는 이진 탐색 트리(BST)에 저장되고, 서버가 시작될 때 파일에서 로드된다. 각 주식 항목은 ID,

재고 수량(left_stock), 가격(price), 읽기 횟수(readcnt), 동기화를 위한 세마포어(mutex)로 구성되며, 이러한 항목들은 노드(node) 구조체로 관리된다. 서버는 멀티스레드 환경을 지원하여 각 클라이언트 연결에 대해 새로운 스레드를 생성하여 요청을 처리한다. 이를 통해 동시에 여러 클라이언트 요청을 병렬로 처리할 수 있다. 세마포어(mutex와 state_mutex)를 사용하여 주식 데이터 접근 및 수정 시 동기화 문제를 방지한다. 특히 readcnt 변수를 통해 여러 읽기 작업이 동시에 이루어질 수 있도록 하고, 쓰기 작업이 진행될 때는 다른 작업이 접근하지 못하도록 한다.

클라이언트는 Task1과 마찬가지로 show, buy <ID> <수량>, sell <ID> <수량> 명령을 보낼 수 있다. show 명령은 모든 주식 항목을 조회하여 클라이언트에게 보내고, buy <ID> <수량> 명령은 특정 주식 항목을 구매하며, 재고 수량이 충분하지 않으면 에러 메시지를 반환한다. sell <ID> <수량> 명령은 특정 주식 항목을 판매한다. 각 명령은 handle_cmd 함수에서 처리되며, 주식 데이터를 수정하는 작업은 process_stock 함수에서 공통적으로 처리된다. 데이터 변경 사항은 즉시 파일에 저장된다.

서버는 시작 시 파일에서 주식 데이터를 로드하고, 클라이언트의 연결을 대기한다. 새로운 클라이언트가 접속하면 스레드를 생성하여 클라이언트의 명령을 처리힌다. 클라이언트가 연결을 종료하면 해당 스레드는 종료되고, 서버는 다른 클라이언트 요청을 계속 처리한다. 모든 클라이언트가 연결을 종료하면 주식 데이터는 파일에 저장된다. 이로써 서버는 안정적으로 주식 데이터를 관리하고, 여러 클라이언트와의 동시 접속을 처리하며, 모든 명령에 대한 응답을 클라이언트에게 제공하여 실시간으로 주식의 조회, 구매, 판매를 수행할 수 있다.

3. 성능 평가 결과 (Task 3)

- 강의자료 슬라이드의 내용 참고하여 작성 (측정 시점, 출력 결과 값 캡처 포함)

1. show, buy, sell의 모든 명령어를 실행시키는 경우

- **측정 시점**

```c
/*elapsed time 측정*/
struct timeval start;
struct timeval end;
unsigned long e_usec;

int main(int argc, char **argv)
{
    gettimeofday(&start, 0); // 시작
    pid_t pids[MAX_CLIENT];
    int runprocess = 0, status, i;
```

```c

    /*elapsed time 측정*/
    gettimeofday(&end, 0); // 종료
    e_usec = ((end.tv_sec * 1000000) + end.tv_usec) - ((start.tv_sec * 1000000) + start.tv_usec);
    printf("elapsed time: %lu microseconds\n", e_usec);
    /*elapsed time 측정*/
    return 0;
}
```

- **출력 결과 값**

(Client 개수 (1개 / 10개/ 50개/ 100개/ 200개/ 300개)에 따라 분류)

Task1

[1개]

```
[buy] success
No such stock ID
1 45 1000
2 190 20000
3 10 1200
4 254 5000
5 104 3700
No such stock ID
[buy] success
elapsed time: 10056810 microseconds
```

[10개]

```
No such stock ID
[buy] success
[buy] success
1 39 1000
2 169 20000
3 11 1200
4 273 5000
5 68 3700
No such stock ID
elapsed time: 10392227 microseconds
```

[50 개]

```
1 106 1000
2 82 20000
3 101 1200
4 155 5000
5 196 3700
[sell] success
[sell] success
No such stock ID
No such stock ID
elapsed time: 10853108 microseconds
```

[100 개]

```
4 146 5000
5 193 3700
No such stock ID
1 146 1000
2 63 20000
3 114 1200
4 146 5000
5 193 3700
No such stock ID
elapsed time: 11269858 microseconds
```

[200개]

```
1 244 1000
2 93 20000
3 217 1200
4 97 5000
5 232 3700
No such stock ID
No such stock ID
No such stock ID
elapsed time: 17469784 microseconds
```

[300개]

```
[sell] success
No such stock ID
No such stock ID
1 444 1000
2 51 20000
3 94 1200
4 122 5000
5 550 3700
[buy] success
elapsed time: 29910065 microseconds
```

Task2

[1개]

```
No such stock ID
No such stock ID
[buy] success
No such stock ID
1 20 1000
2 180 20000
3 18 1200
4 105 5000
5 150 3700
elapsed time: 10116417 microseconds
```

[10개]

```
4 102 5000
5 131 3700
[sell] success
1 9 1000
2 197 20000
3 5 1200
4 102 5000
5 131 3700
[sell] success
elapsed time: 10484558 microseconds
```

[50개]

```
4 140 5000
5 100 3700
No such stock ID
No such stock ID
1 16 1000
2 204 20000
3 15 1200
4 140 5000
5 100 3700
elapsed time: 10802074 microseconds
```

[100개]

```
2 142 20000
3 81 1200
4 20 5000
5 72 3700
[sell] success
[buy] success
No such stock ID
[sell] success
No such stock ID
elapsed time: 12178860 microseconds
```

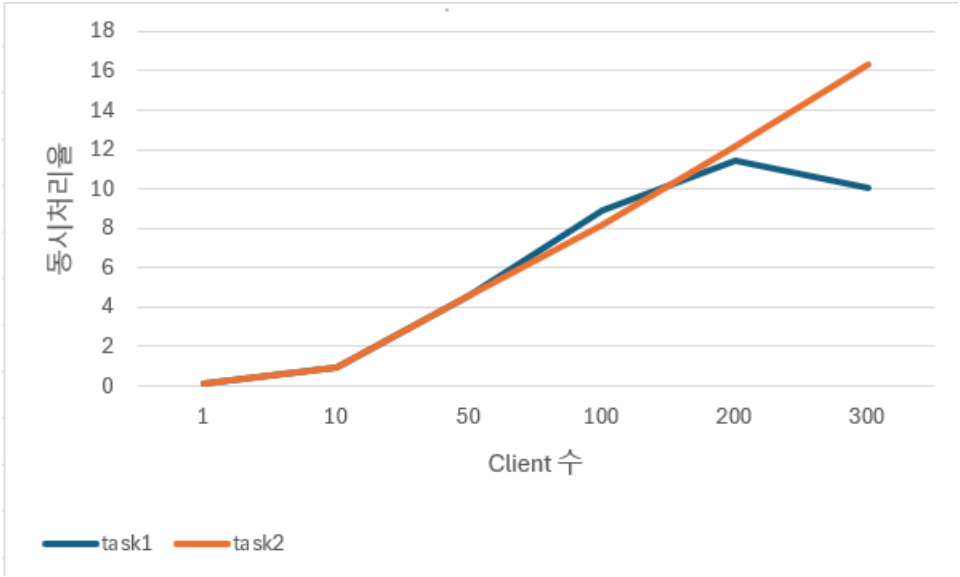
[200개]

```
Not enough left stock
No such stock ID
[buy] success
[sell] success
1 125 1000
2 111 20000
3 135 1200
4 64 5000
5 1 3700
elapsed time: 16386274 microseconds
```

[300개]

```
2 20 20000
3 57 1200
4 64 5000
5 95 3700
1 167 1000
2 20 20000
3 57 1200
4 64 5000
5 95 3700
elapsed time: 18360146 microseconds
```

- **동시처리율 그래프**



- **분석**

Task1 (이벤트 기반 접근 방식)은 클라이언트 수가 증가할수록 동시 처리율이 꾸준히 증가하다가 200 명 이후부터는 감소하는 경향을 보인다. 이는 이벤트 기반 접근 방식이 I/O 바운드 작업을 효율적으로 처리하지만, 클라이언트 수가 너무 많아지면 이벤트 큐의 오버헤드가 발생할 수 있음을 보여준다. 반면, Task2 (스레드 기반 접근 방식)는 클라이언트 수가 증가할수록 동시 처리율이 지속적으로 증가하며, 300 명일 때 가장 높은 처리율을 보인다. 이는 스레드 기반 접근 방식이 더 많은 클라이언트를 효과적으로 처리할 수 있음을 보여준다.

이번 측정은 show, buy, sell 명령어에 대한 동시 처리율을 모두 포함했다. show 명령은 주로 읽기 작업이고, buy 와 sell 명령은 쓰기 작업이다. 이벤트 기반 접근 방식(Task1)은 주로 I/O 바운드 작업에 적합하기 때문에, 혼합된 워크로드에서도 초기에는 높은 성능을 유지할 수 있다. 그러나 클라이언트 수가 증가함에 따라, 쓰기 작업에서의 오버헤드가 누적되어 성능이 저하되는 것을 볼 수 있다. 반면, 스레드 기반 접근 방식(Task2)은 컨텍스트 스위칭 오버헤드가 있지만, 여러 작업을 병렬로 처리할 수 있기 때문에 많은 클라이언트를 효과적으로 처리할 수 있다.

성능 비교를 보면, Task1 은 클라이언트 수가 200 명까지는 안정적인 성능을 보이지만, 그 이후로는 성능이 감소한다. 이는 이벤트 기반 시스템이 특정 임계점을 넘어서면 오버헤드가 증가할 수 있음을 의미한다. 반면, Task2 는 클라이언트 수가 증가할수록 처리율이 꾸준히 증가하며, 300 명일 때 최대 처리율을 기록했다. 이는 스레드 기반 접근 방식이 더 많은 자원을 활용하여 병렬 처리를 효율적으로 수행할 수 있음을 나타낸다.

결론적으로, Task1 (이벤트 기반 접근 방식)은 초기 클라이언트 수에서는 우수한 성능을 보이지만, 클라이언트 수가 많아질수록 성능이 감소한다. 이는 이벤트 기반 시스템의 오버헤드 때문일 수 있다. Task2 (스레드 기반 접근 방식)은 클라이언트 수가 증가할수록 안정적으로 높은 동시 처리율을 유지하며, 최대 300 명 클라이언트까지 효과적으로 처리할 수 있다. 이는 많은 클라이언트를 동시에 처리해야 하는 환경에서 더 적합할 수 있다. 각 접근 방식의 장단점을 고려하여, 상황에 맞는 최적의 방식을 선택함으로써 시스템의 성능을 최적화할 수 있을 것이다.

2. buy, sell의 명령어만 실행시키는 경우

- **측정 시점**

```c
/*elapsed time 측정*/
struct timeval start;
struct timeval end;
unsigned long e_usec;

int main(int argc, char **argv)
{
    gettimeofday(&start, 0); // 시작
    pid_t pids[MAX_CLIENT];
    int runprocess = 0, status, i;
```

```c

    /*elapsed time 측정*/
    gettimeofday(&end, 0); // 종료
    e_usec = ((end.tv_sec * 1000000) + end.tv_usec) - ((start.tv_sec * 1000000) + start.tv_usec);
    printf("elapsed time: %lu microseconds\n", e_usec);
    /*elapsed time 측정*/
    return 0;
}
```

- **출력 결과 값**

(Client 개수 (1개 / 10개/ 50개/ 100개/ 200개/ 300개)에 따라 분류)

Task1

[1 개]

```
No such stock ID
No such stock ID
[sell] success
No such stock ID
[sell] success
No such stock ID
No such stock ID
[buy] success
[buy] success
elapsed time: 10119266 microseconds
```

[10 개]

```
No such stock ID
[buy] success
No such stock ID
No such stock ID
[buy] success
No such stock ID
[sell] success
No such stock ID
No such stock ID
elapsed time: 10288417 microseconds
```

[50 개]

```
No such stock ID
No such stock ID
No such stock ID
No such stock ID
[sell] success
[buy] success
No such stock ID
No such stock ID
No such stock ID
elapsed time: 10787847 microseconds
```

[100 개]

```
No such stock ID
No such stock ID
[buy] success
No such stock ID
No such stock ID
[buy] success
No such stock ID
No such stock ID
[buy] success
elapsed time: 17206837 microseconds
```

[200 개]

```
No such stock ID
No such stock ID
No such stock ID
No such stock ID
No such stock ID
No such stock ID
No such stock ID
No such stock ID
[buy] success
elapsed time: 30101887 microseconds
```

[300 개]

```
No such stock ID
No such stock ID
No such stock ID
No such stock ID
No such stock ID
No such stock ID
[buy] success
No such stock ID
No such stock ID
elapsed time: 34363050 microseconds
```

Task2

[1 개] [10 개]

```
No such stock ID
[buy] success
[buy] success
[buy] success
[sell] success
No such stock ID
No such stock ID
No such stock ID
No such stock ID
elapsed time: 10072280 microseconds
```

```
No such stock ID
[buy] success
[sell] success
[buy] success
No such stock ID
[buy] success
[buy] success
No such stock ID
No such stock ID
elapsed time: 10158156 microseconds
```

[50 개]

```
[buy] success
No such stock ID
No such stock ID
No such stock ID
[buy] success
No such stock ID
[buy] success
[sell] success
[buy] success
elapsed time: 12187042 microseconds
```

[100 개]

```
[buy] success
[buy] success
No such stock ID
No such stock ID
No such stock ID
[buy] success
No such stock ID
[buy] success
No such stock ID
elapsed time: 11171684 microseconds
```

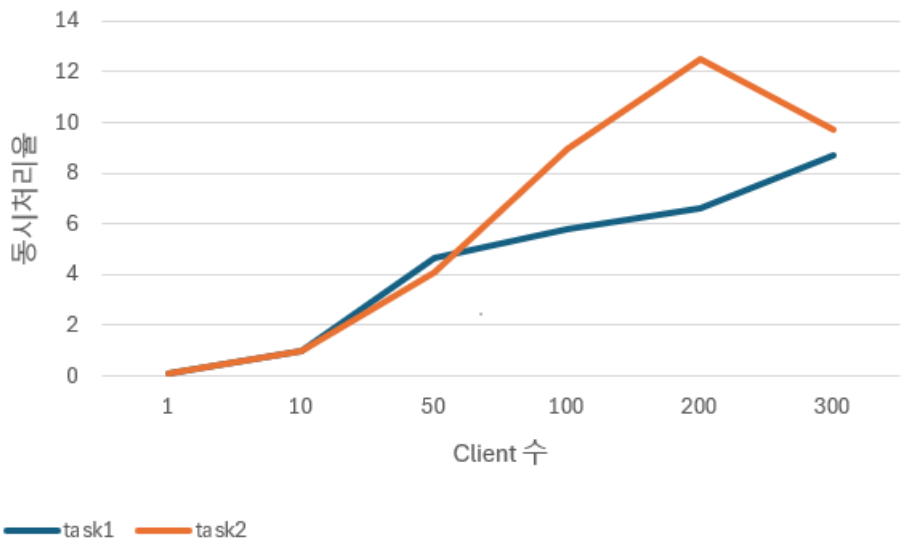
[200 개]

```
No such stock ID
No such stock ID
No such stock ID
No such stock ID
No such stock ID
[buy] success
No such stock ID
No such stock ID
No such stock ID
elapsed time: 15998367 microseconds
```

[300 개]

```
[buy] success
No such stock ID
[sell] success
No such stock ID
[sell] success
[sell] success
[buy] success
[buy] success
[sell] success
elapsed time: 30831722 microseconds
```

- **동시처리율 그래프**



- **분석**

Task1 은 클라이언트 수가 증가할수록 꾸준히 성능이 향상되어, 300 명까지도 높은 동시 처리율을 유지한다. 이는 이벤트 기반 시스템이 I/O 바운드 작업에 적합하며, 많은 클라이언트를 효율적으로 처리할 수 있음을 보여준다. 반면, Task2 는 클라이언트 수가 200 명일 때 최대 성능을 보이지만, 그 이후에는 자원 경쟁과 컨텍스트 스위칭 오버헤드로 인해 성능이 저하되는 것으로 보인다. 이는 스레드 기반 접근 방식이 초기에는 우수한 성능을 보이지만, 많은 클라이언트를 처리할 때는 한계가 있음을 나타낸다.

결론적으로, Task1(이벤트 기반 접근 방식)은 클라이언트 수가 증가할수록 꾸준히 높은 동시 처리율을 유지하여 많은 클라이언트를 동시에 처리하는 데 적합하다. Task2(스레드 기반 접근 방식)는 초기에는 높은 성능을 보이지만, 200 명 이후에는 성능이 저하된다. 이는 스레드 간의 자원 경쟁과 오버헤드 때문일 수 있다.

3. show 명령어만 실행시키는 경우

- **측정 시점**

```c
/*elapsed time 측정*/
struct timeval start;
struct timeval end;
unsigned long e_usec;

int main(int argc, char **argv)
{
    gettimeofday(&start, 0); // 시작
    pid_t pids[MAX_CLIENT];
    int runprocess = 0, status, i;
```

```c

    /*elapsed time 측정*/
    gettimeofday(&end, 0); // 종료
    e_usec = ((end.tv_sec * 1000000) + end.tv_usec) - ((start.tv_sec * 1000000) + start.tv_usec);
    printf("elapsed time: %lu microseconds\n", e_usec);
    /*elapsed time 측정*/
    return 0;
}
```

- **출력 결과 값**

(Client 개수 (1개 / 10개/ 50개/ 100개/ 200개/ 300개)에 따라 분류)

Task1

[1 개]

```
2 120 20000
3 172 1200
4 271 5000
5 175 3700
1 170 1000
2 120 20000
3 172 1200
4 271 5000
5 175 3700
elapsed time: 10005027 microseconds
```

[10 개]

```
2 120 20000
3 172 1200
4 271 5000
5 175 3700
1 170 1000
2 120 20000
3 172 1200
4 271 5000
5 175 3700
elapsed time: 10005906 microseconds
```

[50 개]

```
2 120 20000
3 172 1200
4 271 5000
5 175 3700
1 170 1000
2 120 20000
3 172 1200
4 271 5000
5 175 3700
elapsed time: 10018991 microseconds
```

[100 개]

```
2 120 20000
3 172 1200
4 271 5000
5 175 3700
1 170 1000
2 120 20000
3 172 1200
4 271 5000
5 175 3700
elapsed time: 10033396 microseconds
```

[200 개]

```
2 120 20000
3 172 1200
4 271 5000
5 175 3700
1 170 1000
2 120 20000
3 172 1200
4 271 5000
5 175 3700
elapsed time: 11022171 microseconds
```

[300 개]

```
2 120 20000
3 172 1200
4 271 5000
5 175 3700
1 170 1000
2 120 20000
3 172 1200
4 271 5000
5 175 3700
elapsed time: 11231931 microseconds
```

Task2

[1 개]

```
2 222 20000
3 132 1200
4 175 5000
5 67 3700
1 72 1000
2 222 20000
3 132 1200
4 175 5000
5 67 3700
elapsed time: 10003814 microseconds
```

[10 개]

```
2 222 20000
3 132 1200
4 175 5000
5 67 3700
1 72 1000
2 222 20000
3 132 1200
4 175 5000
5 67 3700
elapsed time: 10005783 microseconds
```

[50 개]

```
2 222 20000
3 132 1200
4 175 5000
5 67 3700
1 72 1000
2 222 20000
3 132 1200
4 175 5000
5 67 3700
elapsed time: 10010354 microseconds
```

[100 개]

```
2 222 20000
3 132 1200
4 175 5000
5 67 3700
1 72 1000
2 222 20000
3 132 1200
4 175 5000
5 67 3700
elapsed time: 10018152 microseconds
```

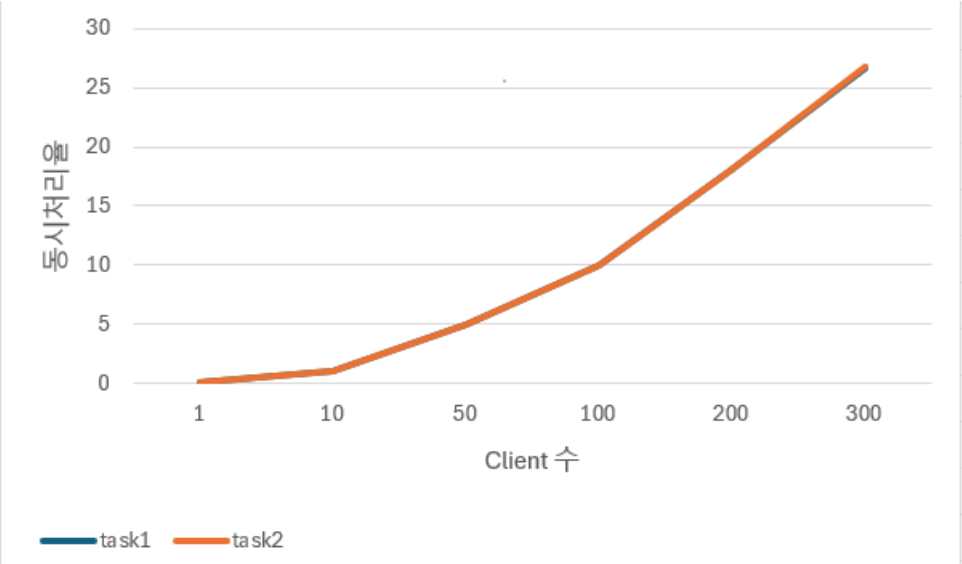
[200 개]

```
2 222 20000
3 132 1200
4 175 5000
5 67 3700
1 72 1000
2 222 20000
3 132 1200
4 175 5000
5 67 3700
elapsed time: 11015257 microseconds
```

[300 개]

```
2 222 20000
3 132 1200
4 175 5000
5 67 3700
1 72 1000
2 222 20000
3 132 1200
4 175 5000
5 67 3700
elapsed time: 11218431 microseconds
```

- **동시처리율 그래프**



- **분석**

종합적으로 분석하면, Task1(이벤트 기반 접근 방식)과 Task2(스레드 기반 접근 방식) 모두 클라이언트 수가 증가할수록 동시 처리율이 선형적으로 증가하는 모습을 보인다. 이는 두 접근 방식 모두 show 명령어와 같은 읽기 작업에 대해 높은 확장성을 갖추고 있음을 보여준다. Task1 과 Task2 의 성능은 매우 유사하며, 클라이언트 수가 증가함에 따라 거의 동일한 처리율을 보인다. 이는 show 명령어가 주로 읽기 작업으로 이루어져 있어, 이벤트 기반 접근 방식과 스레드 기반 접근 방식 모두 효율적으로 처리할 수 있음을 보여준다. 효율성 측면에서 이벤트 기반 접근 방식(Task1)은 비동기 I/O 를 통해 자원 소비를 최소화하며, 많은 클라이언트의 요청을 효과적으로 처리할 수 있다. 반면, 스레드 기반 접근 방식(Task2)은 각 요청을 별도의 스레드로 처리하여 높은 병렬 처리 능력을 발휘한다. 결론적으로, Task1(이벤트 기반 접근 방식)은 클라이언트 수가 증가할수록 꾸준히 높은 동시 처리율을 유지하며, 많은 클라이언트의 show 명령어를 효율적으로 처리할 수 있다. Task2(스레드 기반 접근 방식) 역시 클라이언트 수가 증가할수록 꾸준히 높은 동시 처리율을 유지하며, 병렬 처리를 통해 많은 클라이언트의 요청을 효과적으로 처리할 수 있다. 두 접근 방식 모두 show 명령어와 같은 읽기 작업에 대해 높은 성능과 확장성을 보이므로, 특정 상황에 따라 최적의 방식을 선택함으로써 시스템의 성능을 최적화할 수 있을 것이다.